

# NYCU-ECE DCS-2026

## HW02

### Design: Sobel Edge Detection

#### Data Preparation

---

1. Extract the files needed from TA's directory:

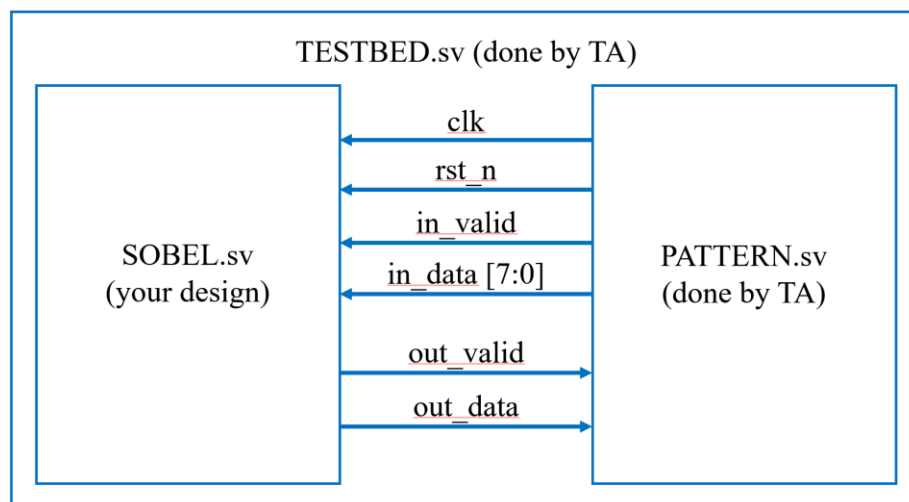
```
% tar -xvf ~dcsTA01/HW02.tar
```

2. The extracted files contain:

- a. 00\_TESTBED/
- b. 01\_RTL/
- c. 02\_SYN/
- d. 03\_GATE/
- e. 09\_UPLOAD/

#### Block Diagram

---



#### Design Description

---

The Sobel operator is a widely used algorithm in image processing and computer vision for edge detection. It works by calculating the gradient of image intensity at each pixel, which highlights areas where brightness changes rapidly—typically indicating the boundaries of objects. By convolving the image with two 3x3 kernels (one for horizontal changes and one for vertical), it produces a gradient magnitude that emphasizes edges while providing a small amount of noise smoothing.

In this homework, you are asked to implement a Sobel Edge Detection Accelerator, which processes a stream of 8-bit grayscale pixel values from a fixed-size 8×8 image (64 pixels total). Your circuit must compute the Sobel gradient magnitude and decide whether that pixel is an edge.

## Algorithm Overview

The Sobel operator convolves a 3×3 neighborhood with two kernels to estimate the horizontal gradient  $G_x$  and vertical gradient  $G_y$  :

|       |    |   |   |  |       |    |    |    |
|-------|----|---|---|--|-------|----|----|----|
| $G_x$ |    |   |   |  | $G_y$ |    |    |    |
|       | -1 | 0 | 1 |  |       | -1 | -2 | -1 |
|       | -2 | 0 | 2 |  |       | 0  | 0  | 0  |
|       | -1 | 0 | 1 |  |       | 1  | 2  | 1  |

The gradient magnitude is approximated as  $|G_x| + |G_y|$ . If the result exceeds a fixed threshold  $T = 128$ , the output pixel is labelled **1** (edge); otherwise **0** (non-edge).

For example:

|       |     |     |     |     |     |     |     |     |
|-------|-----|-----|-----|-----|-----|-----|-----|-----|
| input |     |     |     |     |     |     |     |     |
|       | 156 | 155 | 98  | 30  | 203 | 205 | 133 | 137 |
|       | 176 | 183 | 96  | 183 | 221 | 167 | 9   | 224 |
|       | 108 | 23  | 178 | 227 | 110 | 248 | 110 | 38  |
|       | 111 | 241 | 7   | 114 | 189 | 13  | 199 | 13  |
|       | 121 | 65  | 140 | 195 | 159 | 215 | 121 | 167 |
|       | 51  | 46  | 54  | 148 | 147 | 165 | 157 | 227 |
|       | 43  | 217 | 44  | 148 | 4   | 43  | 51  | 165 |
|       | 7   | 19  | 213 | 184 | 53  | 76  | 32  | 29  |

$$|G_x| = |156 \cdot -1 + 176 \cdot -2 + 108 \cdot -1 + 98 \cdot 1 + 96 \cdot 2 + 178 \cdot 1| = 148$$

$$|G_y| = |156 \cdot -1 + 155 \cdot -2 + 98 \cdot -1 + 108 \cdot 1 + 23 \cdot 2 + 178 \cdot 1| = 232$$

$$|G_x| + |G_y| > 128, \text{ out\_data} = 1 \text{ (center pixel[1][1])}$$

|       |     |     |     |     |     |     |     |     |
|-------|-----|-----|-----|-----|-----|-----|-----|-----|
| input |     |     |     |     |     |     |     |     |
|       | 156 | 155 | 98  | 30  | 203 | 205 | 133 | 137 |
|       | 176 | 183 | 96  | 183 | 221 | 167 | 9   | 224 |
|       | 108 | 23  | 178 | 227 | 110 | 248 | 110 | 38  |
|       | 111 | 241 | 7   | 114 | 189 | 13  | 199 | 13  |
|       | 121 | 65  | 140 | 195 | 159 | 215 | 121 | 167 |
|       | 51  | 46  | 54  | 148 | 147 | 165 | 157 | 227 |
|       | 43  | 217 | 44  | 148 | 4   | 43  | 51  | 165 |
|       | 7   | 19  | 213 | 184 | 53  | 76  | 32  | 29  |

$$|G_x| = |155 \cdot -1 + 183 \cdot -2 + 23 \cdot -1 + 30 \cdot 1 + 183 \cdot 2 + 227 \cdot 1| = 79$$

$$|G_y| = |155 \cdot -1 + 98 \cdot -2 + 30 \cdot -1 + 23 \cdot 1 + 178 \cdot 2 + 227 \cdot 1| = 225$$

$$|G_x| + |G_y| > 128, \text{ out\_data} = 1 \text{ (center pixel[1][2])}$$

...

|       |     |     |     |     |     |     |     |     |
|-------|-----|-----|-----|-----|-----|-----|-----|-----|
| input |     |     |     |     |     |     |     |     |
|       | 156 | 155 | 98  | 30  | 203 | 205 | 133 | 137 |
|       | 176 | 183 | 96  | 183 | 221 | 167 | 9   | 224 |
|       | 108 | 23  | 178 | 227 | 110 | 248 | 110 | 38  |
|       | 111 | 241 | 7   | 114 | 189 | 13  | 199 | 13  |
|       | 121 | 65  | 140 | 195 | 159 | 215 | 121 | 167 |
|       | 51  | 46  | 54  | 148 | 147 | 165 | 157 | 227 |
|       | 43  | 217 | 44  | 148 | 4   | 43  | 51  | 165 |
|       | 7   | 19  | 213 | 184 | 53  | 76  | 32  | 29  |

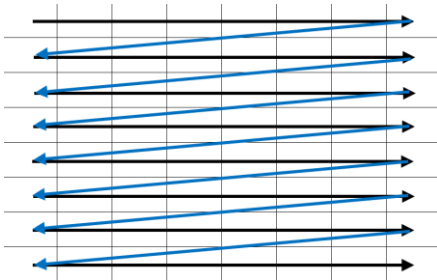
$$|G_x| = |165 \cdot -1 + 43 \cdot -2 + 76 \cdot -1 + 227 \cdot 1 + 165 \cdot 2 + 29 \cdot 1| = 259$$

$$|G_y| = |165 \cdot -1 + 157 \cdot -2 + 227 \cdot -1 + 76 \cdot 1 + 32 \cdot 2 + 29 \cdot 1| = 537$$

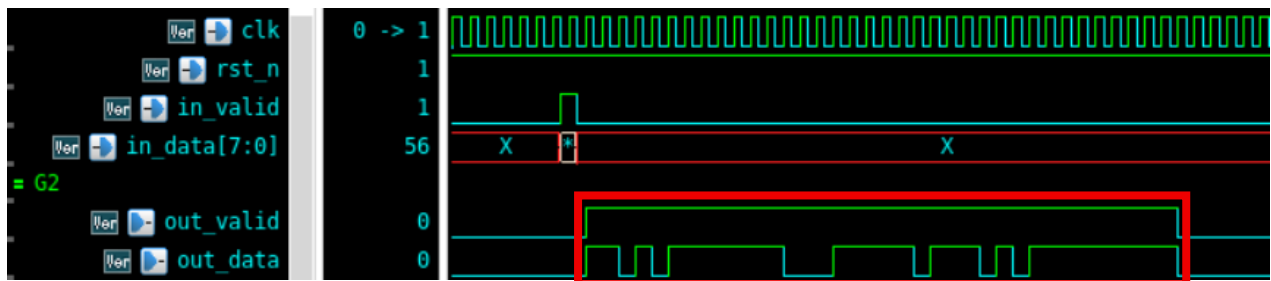
$$|G_x| + |G_y| > 128, \text{ out\_data} = 1 \text{ (center pixel[6][6])}$$

## Additional information

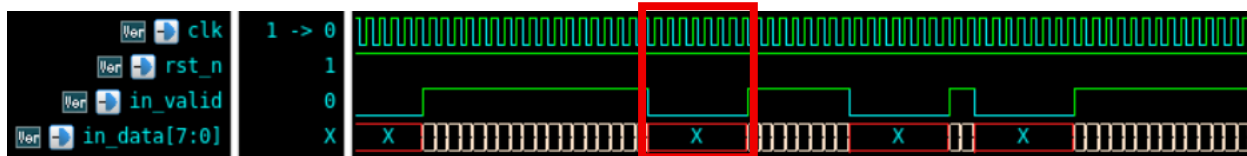
1. The input image pixel is sent in serial through in\_data, in **raster scan** order.



2. Since the inputs are fixed-size 8×8 images, there will be 6×6=36 out\_data for each input image (no padding needed).



3. Each input pixel has a **10% chance to stall**, and every stall takes **8 cycles**.



4. Nonconsecutive output is acceptable, which means the 36 out\_data don't need to be consecutive.



5. When failing a pattern, you can use “hw2\_debug.txt” in 01\_RTL to debug.

## Inputs

| Signal name | Number of bit | Description                                                                                                       |
|-------------|---------------|-------------------------------------------------------------------------------------------------------------------|
| clk         | 1             | 8ns clock signal                                                                                                  |
| rst_n       | 1             | Asynchronous active-low reset signal                                                                              |
| in_valid    | 1             | Pulled high during pixel input                                                                                    |
| in_data     | 8             | 8-bit pixel value; valid when in_valid is high<br>(may be discontinuous, total 64 cycles).<br>Data range: 0 ~ 255 |

## Outputs

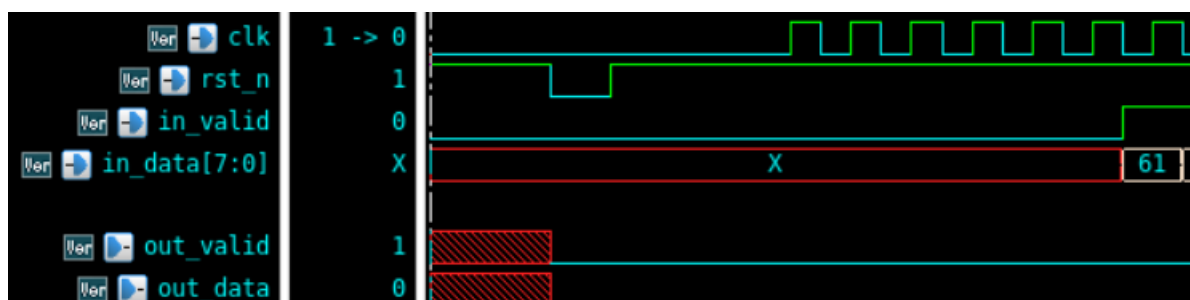
| Signal name | Number of bit | Description                                                                                                            |
|-------------|---------------|------------------------------------------------------------------------------------------------------------------------|
| out_valid   | 1             | Pulled high during output<br>(reset required)                                                                          |
| out_data    | 1             | Edge map output: 1 = edge, 0 = non-edge.<br>Output 36 cycles (Discontinuous output is acceptable).<br>(reset required) |

## Specifications

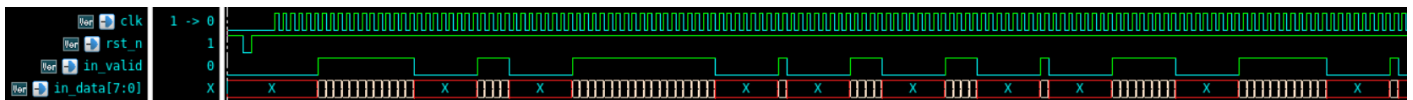
1. Top module name: **SOBEL**(File name : **SOBEL.sv**)
2. After asynchronous active-low reset, all output signals must be reset to zero.
3. **in\_valid and out\_valid cannot overlap.**
4. Output must be produced within 200 cycles after the last input pixel is provided.
5. Output must be exactly 36 cycles, no more, no less.
6. The next input will be sent 2-4 cycles after last out\_valid is pulled down.
7. The 02\_SYN result **must not have any errors** and **must not contain any latches**.
8. The slack in the timing report must be non-negative and the result must be "Met".
9. Gate level simulation **must not have any timing violation**.
10. Input delay = 0.5 \* clock period; Output delay = 0.5 \* clock period.
11. Do not use \*error\*, \*latch\*, \*congratulation\* or \*fail\* as names for logic / wire / reg / submodule / parameter , otherwise the demo result will be fail.  
Note: \* represents any symbol before or after the word. Ex: error\_test is prohibited.
12. Combinational blocks and sequential blocks should be separated.

## Example waveform

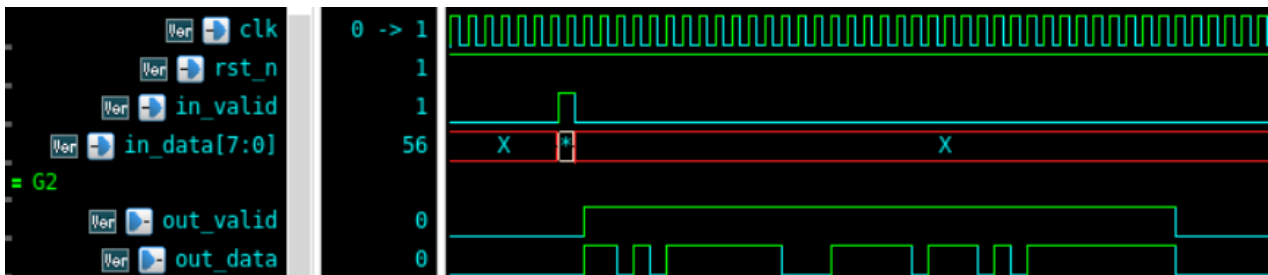
Reset all output after asynchronous and active-low reset signal:



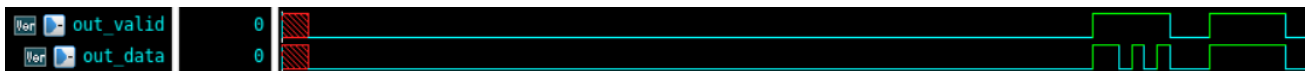
64 cycles of input image pixel with 10% chance of stall, each stall takes 8 cycles:



36 output cycles for each input pattern:



Output can be nonconsecutive:



## Upload file

1. Upload the following files:

| File               | Filename                                         |                                           |
|--------------------|--------------------------------------------------|-------------------------------------------|
| SystemVerilog Code |                                                  | Upload using ./01_upload under 09_UPLOAD/ |
| Report             | hw1_report_dcsXXX.pdf<br>(XXX is server account) | Upload to E3                              |

\* Note that the script requires time to synchronize with the NTP server. Please avoid last-minute submissions.

\* Incorrect filenames will result in a 5-point deduction.

2. Submission time:

| Submit | Deadline         |
|--------|------------------|
| 1de    | 2026/04/02 23:59 |
| 2de    | 2026/04/07 15:30 |

## Grading policy

| Criteria                                          | %  | Description                                         |
|---------------------------------------------------|----|-----------------------------------------------------|
| Pass the RTL, Synthesis and GATE level simulation | 60 | Pass all pattern and no error in synthesis.         |
| Performance                                       | 35 | Ranking formula: <b>total_latency * area</b>        |
| Report                                            | 5  | The report should be concise, not exceeding two A4. |

\*For 2de, total score will get 40% off.

## Note

---

Template folders and reference commands:

1. 01\_RTL/ (RTL simulation) → **./01\_run\_vcs\_rtl**
2. 02\_SYN/ (synthesis) → **./01\_run\_dc**
3. 03\_GATE/ (GATE simulation) → **./01\_run\_vcs\_gate**
4. 09\_UPLOAD/ (upload) → **./01\_upload**